

PERFORM LOGICAL DATA EXTRACTION FROM AN ANDROID DEVICE USING AFLOGICAL OSE

EX. NO: 7

Date :

1. AIM

To establish an Android forensic connection between a Windows workstation and an Android device using Android Debug Bridge (ADB), verify the connected device and its accessible storage, transfer controlled test evidence to the Android device, acquire the transferred data back to the Windows workstation, and verify the presence of the acquired files.

2. SOFTWARE / TOOLS REQUIRED

Item	Details
Operating System	Windows
Acquisition Tool	Android Debug Bridge (ADB)
Platform-Tools Version	Android SDK Platform-Tools 37.0.1
ADB Version	1.0.41
Target Device	Android mobile device
Connection	USB
Evidence Type	Controlled test files

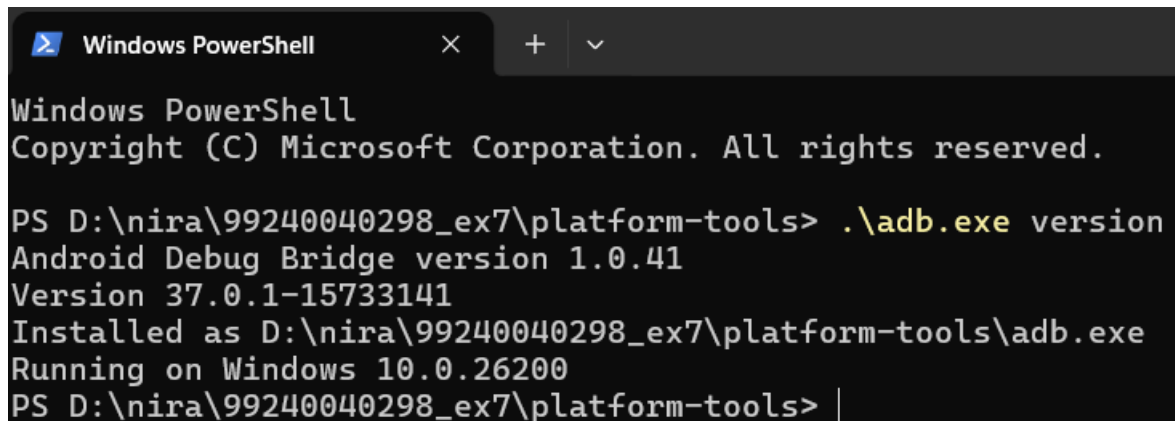
3. THEORY

AFLogical OSE (Open Source Edition, developed by viaForensics) is an application-based tool installed via the Android Debug Bridge (ADB) is a command-line utility provided as part of the Android SDK Platform-Tools. It allows an authorized computer to communicate with an Android device through USB and perform operations such as device identification, shell commands, application management, and controlled file transfer. Logical data extraction is a mobile forensic technique used to collect accessible information from an operating Android device without creating a physical bit-by-bit image of the storage. In this experiment, ADB was used to establish communication with the Android device and perform controlled logical acquisition of test evidence files.

4. PROCEDURE

Step 1: ADB Environment Setup

The Android SDK Platform-Tools were configured on the Windows workstation and the ADB installation was verified.



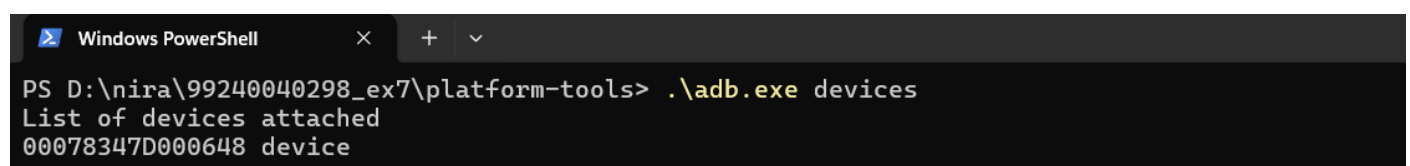
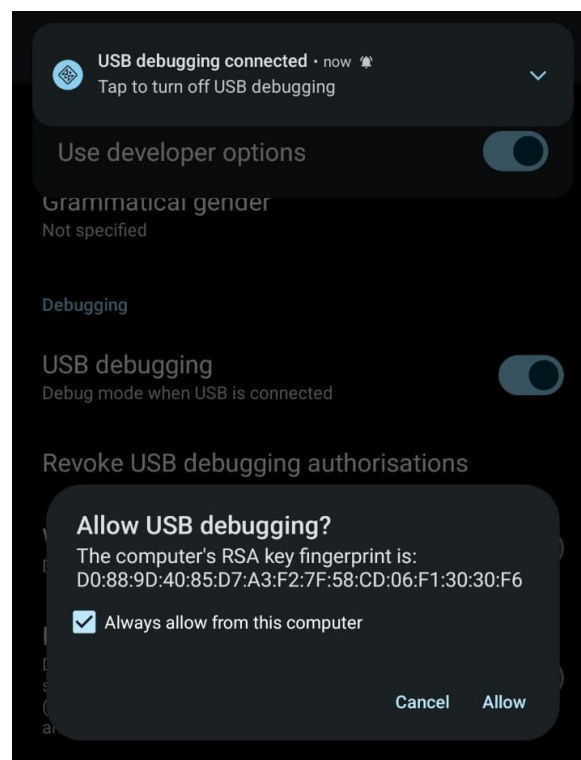
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\nira\99240040298_ex7\platform-tools> .\adb.exe version
Android Debug Bridge version 1.0.41
Version 37.0.1-15733141
Installed as D:\nira\99240040298_ex7\platform-tools\adb.exe
Running on Windows 10.0.26200
PS D:\nira\99240040298_ex7\platform-tools> |
```

Figure 1: Verification of the ADB environment and Platform-Tools installation.

Step 2: Connecting and Verifying the Device

USB debugging was enabled on the Android device and the workstation was authorized. The connected device was verified using the adb devices command.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\nira\99240040298_ex7\platform-tools> .\adb.exe devices
List of devices attached
00078347D000648 device
```

Figure 2: ADB terminal showing the authorized Android device connected to the Windows workstation.

Step 3: Device Identification

The device model and Android version were obtained using ADB shell properties.

```
PS D:\nira\99240040298_ex7\platform-tools> .\adb.exe shell getprop ro.product.model
A142
PS D:\nira\99240040298_ex7\platform-tools> .\adb.exe shell getprop ro.build.version.release
16
```

Figure 3: Identification of the connected device as A142 running Android 16.

Step 4: Storage Verification

The shared storage of the Android device was examined using the ADB shell. The accessible directories, including the Download directory, were verified.

```
PS D:\nira\99240040298_ex7\platform-tools> .\adb.exe shell ls /sdcard/
Alarms
Android
Audiobooks
DCIM
Documents
Download
MX File Transfer
Movies
Music
MxPlayerAd
Notifications
Pictures
Podcasts
Recordings
Ringtones
Saved Searches
Subtitles
mvk
```

Figure 4: Listing of accessible directories in the Android device storage.

Step 5: Transfer and Verification of Test Data

The prepared TestData directory containing the controlled test files was transferred to the Android device using adb push. The transferred files were then verified in the device's Download/TestData directory.

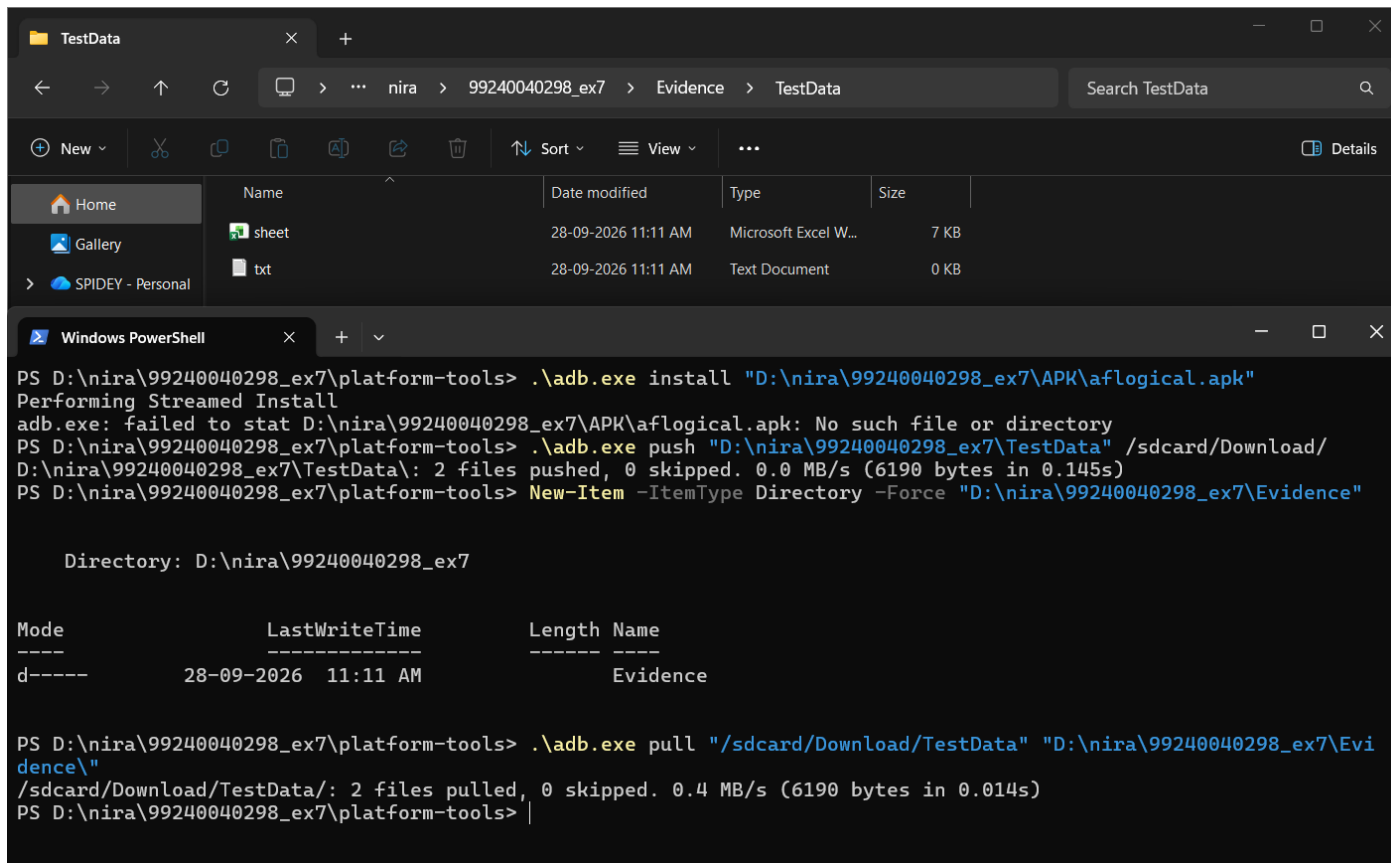


Figure 5: Transfer and verification of the TestData files on the Android device.

Step 6: Logical Acquisition

A dedicated Evidence directory was created on the Windows workstation. The TestData directory was acquired from the Android device using adb pull.

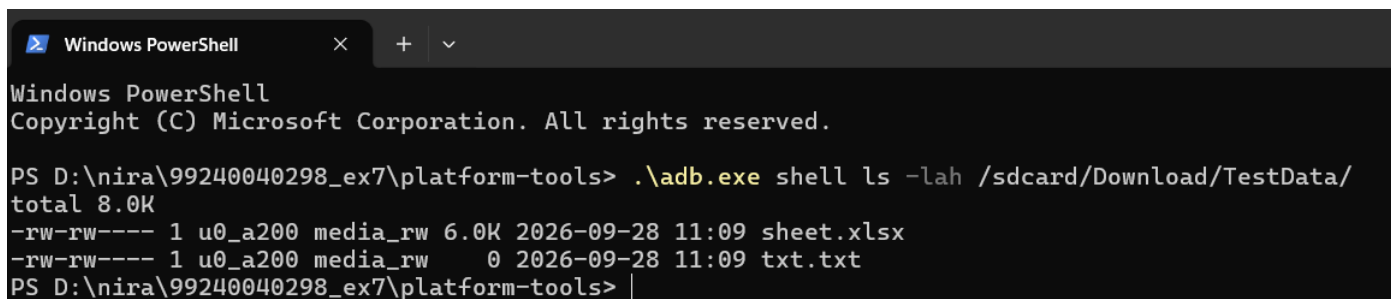


Figure 6: Successful logical acquisition of the TestData directory from the Android device.

Step 7: Evidence Verification

The acquired files were examined in the Windows Evidence directory to confirm successful transfer and preservation of the collected data.

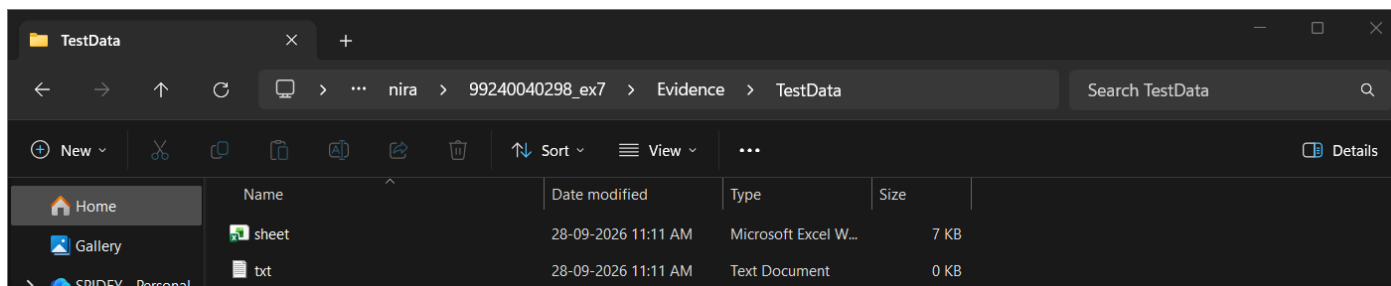


Figure 7: Acquired TestData files displayed in the Windows Evidence directory.

5. OBSERVATIONS / FORENSIC FINDINGS

1. ADB was successfully configured on the Windows workstation.
2. The Android device was successfully detected and authorized.
3. The device was identified as A142, running Android 16.
4. Android shared storage was successfully accessed.
5. Two test files were transferred using adb push.
6. The files were verified on the Android device.
7. Both files were successfully acquired using adb pull.
8. The acquisition reported 2 files pulled and 0 skipped.
9. The acquired files were stored in the designated Evidence folder.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

6.RESULT

Logical acquisition of controlled test evidence from the Android device was successfully performed using ADB. The files sheet.xlsx and txt.txt were transferred to the device, verified, acquired back to the Windows workstation, and stored successfully in the Evidence folder.